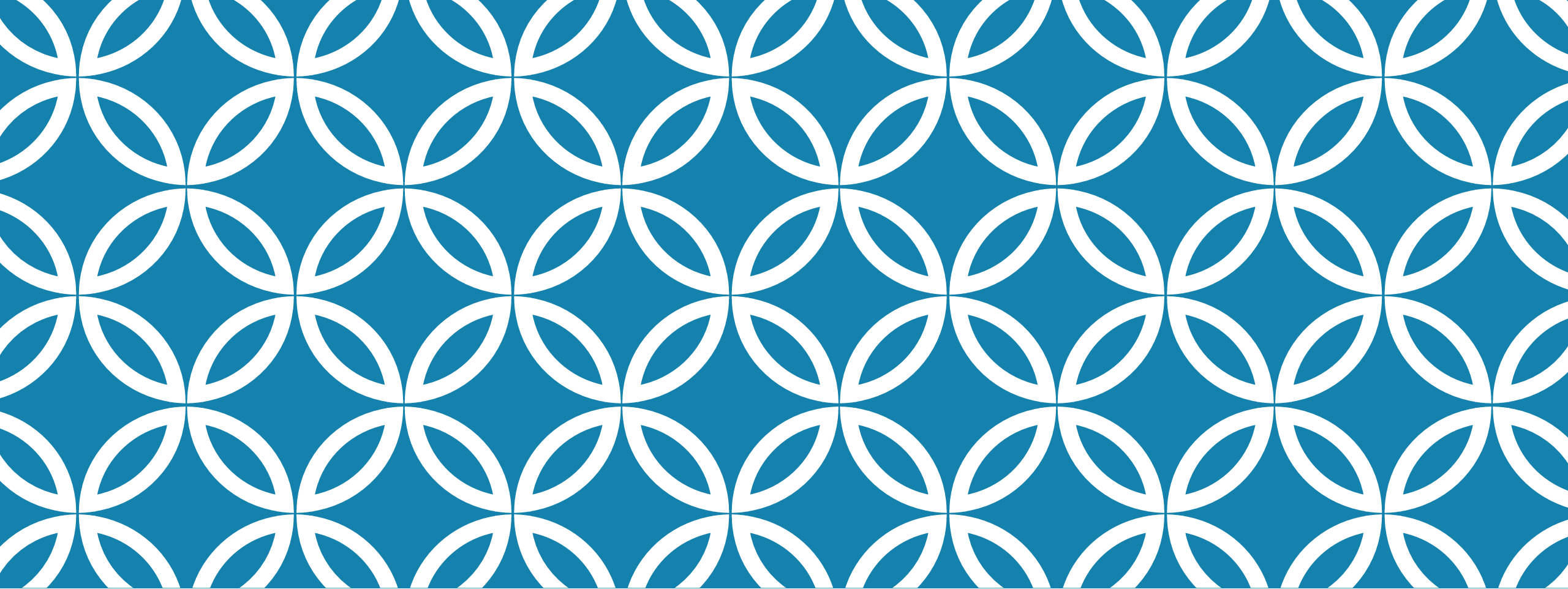




STATIC PROGRAM ANALYSIS

Lecture 19 – Infer: identifying
sensitivity levels

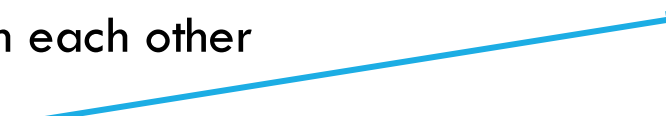
OPEN-SOURCE ANALYZERS

Very user oriented

- Identify set of bugs
- One analysis focuses on a certain type of bugs
- No clear how these analysis interact with each other
- Infer has default checkers for analyze

Cannot ask for invariants to determine sensitivity level

- Is this branch outcome feasible/infeasible?
- What is a CG? Does it create two contexts for a method?
- Handling assertions $\Leftrightarrow$ branch sensitivity.
- We can only “query” by the type of bugs they can detect, e.g., NullPointerException dereference such it is the main starting point of infer (next class: separation logic).



- biabduction
- fragment-retains-view
- inefficient-keyset-iterator
- linters
- liveness
- racerd
- dotnet-resource-leak
- siof
- self-in-block
- starvation
- uninit

FLOW SENSITIVITY LEVELS

Flow sensitivity

Branch sensitivity

Path sensitivity



Totally ordered

How to determine the sensitive level?

Flow sensitive: can differentiate two assignments to the same variable?

Branch sensitivity: can reason about the branching point to determine flow facts?

Path sensitivity: can determine that a particular combination of true/false is not feasible.

```
if (x > y) {  
  x = x + y;  
  y = x - y;  
  x = x - y;  
  if (x > y)  
    assert false;  
}
```



CONTEXT SENSITIVITY (NO TYPES)

Static methods

Compile-time accurate call graphs

How can we check that it is context sensitive?

Combining information decreases accuracy, hence could produce warnings.

TYPE SENSITIVITY CG

Recall CG in OO depends on variable type

RA – reachability

CHA – class hierarchy

RTA – rapid type

Field, Method, Exception variations (MTA, MTA, ETA)

0-CFA, 1-CFA – uses points-to analysis, thus more precise than RTA



Totally ordered

RA

Fully resolved at run-time

At compile time one call site could match several method definitions

`void m(Object o){o.toString()};` `o.toString()` matches all method definitions in all classes that match `String toString()`

Naïve reachability analysis (RA) is sound, but grossly imprecise

CHA

Consider the type hierarchy

- Look at the declared type of a variable (target, receiver object)
- Restrict only to methods found in subtype of that type
- More precise than RA, and not that computationally expensive

Some types in the class hierarchy might never be instantiated/allocated

False positives

RTA

A dispatch should be done on objects that are allocated (e.g., new ...)

Start with CHA CG

Construct a new CG

- Start with main
- Add only those edges for which types are allocated
- Methods that are reachable are added to the method processing queue
- Consider supertypes of the allocated types
- Methods can be re-evaluated several times since a different parameter type could be passed to them

RTA ALGORITHM

More precise graph than CHA

- RTA does not add edges from CHA that are not feasible at runtime

Not sound

$T \leftarrow \text{set of allocated types in } m$

$T \leftarrow T \cup \text{allocated types in RTA callers of } M$

Unsoundness, only parents relations

```
public static void main(String[] args){
    Object o = foo();
    bar(o);
}

public static Object foo(){
    return new A();
}

public static void bar(Object o){
    o.toString()
}
```

```
public static Object v;

public static void main(String[] args){
    foo();
    bar();
}

public static void foo(){
    Object o = new A();
    v = o;
}

public static void bar(){
    v.toString()
}
```

Since neither *main* nor *bar* allocate any objects
 $T = \emptyset$ when invoking *bar* from *main*

DETECTING TYPE SENSITIVITY

How can we use null pointer dereference to understand the sensitivity?

OBJECT SENSITIVITY

Escape analysis are used to determine whether null can “escape” the method.